

LABSHEET 8

- Step 1: Import the required libraries and load a real-world video sequence or capture live video using a webcam.

```
import cv2
import matplotlib.pyplot as plt

from google.colab import files
uploaded = files.upload()
filename = next(iter(uploaded))

cap = cv2.VideoCapture(filename)
print("Video loaded successfully.")

print("Name: Aryan Agarwal")
print("CU240251800")
```

Choose Files 14775656_..._30fps.mp4
14775656_1920_1080_30fps.mp4(video/mp4) - 25901462 bytes, last modified: 9/10/2026 - 100% done
 Saving 14775656_1920_1080_30fps.mp4 to 14775656_1920_1080_30fps (2).mp4
 Video loaded successfully.
 Name: Aryan Agarwal
 CU240251800

- Step 2: Identify a moving object or select feature points using the Shi-Tomasi Corner Detection algorithm.

```
ret, frame = cap.read()

gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

corners = cv2.goodFeaturesToTrack(
    gray, 30, 0.01, 10
)

for c in corners:
    x, y = c.ravel().astype(int)
    cv2.circle(frame, (x, y), 4, (0, 255, 0), -1)

plt.figure(figsize=(4, 3))
plt.imshow(cv2.cvtColor(frame, cv2.COLOR_BGR2RGB))
plt.title("Shi-Tomasi Corners")
plt.axis("off")
plt.show()
plt.close()

print("Name: Aryan Agarwal")
print("CU240251800")
```

Shi-Tomasi Corners



Name: Aryan Agarwal
 CU240251800

Step 3 Apply the Lucas-Kanade Optical Flow algorithm to track the movement of the selected object across consecutive video frames.

```
cap.set(cv2.CAP_PROP_POS_FRAMES, 0)

ret, old = cap.read()
ret, new = cap.read()

old_gray = cv2.cvtColor(old, cv2.COLOR_BGR2GRAY)
new_gray = cv2.cvtColor(new, cv2.COLOR_BGR2GRAY)

p0 = cv2.goodFeaturesToTrack(old_gray, 30, 0.01, 10)

p1, status, _ = cv2.calcOpticalFlowPyrLK(
    old_gray, new_gray, p0, None
)

print("Lucas-Kanade tracking completed.")

print("Name: Aryan Agarwal")
print("CU240251800")
```

```
Lucas-Kanade tracking completed.
Name: Aryan Agarwal
CU240251800
```

Step 4 Draw motion trajectories and displacement vectors to visualize the path followed by the moving object.

```
output = new.copy()

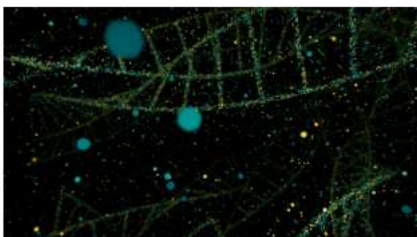
for a, b, s in zip(p0, p1, status):
    if s == 1:
        x1, y1 = a.ravel().astype(int)
        x2, y2 = b.ravel().astype(int)

        cv2.arrowedLine(output, (x1,y1), (x2,y2), (0,255,0), 2)

plt.figure(figsize=(4, 3))
plt.imshow(cv2.cvtColor(output, cv2.COLOR_BGR2RGB))
plt.title("Motion Vectors")
plt.axis("off")
plt.show()
plt.close()

print("Name: Aryan Agarwal")
print("CU240251800")
```

Motion Vectors



```
Name: Aryan Agarwal
CU240251800
```

Step 5 Calculate the direction and magnitude of object motion and display the tracking results in real time.

```
import numpy as np

for a, b, s in zip(p0, p1, status):
```

```

if s == 1:
    dx, dy = b.ravel() - a.ravel()
    magnitude = np.sqrt(dx**2 + dy**2)
    direction = np.degrees(np.arctan2(dy, dx))

    print("Magnitude:", round(magnitude, 2),
          "Direction:", round(direction, 2), "degrees")

print("Name: Aryan Agarwal")
print("CU240251800")

```

```

Magnitude: 12.04 Direction: -105.63 degrees
Magnitude: 8.71 Direction: 136.42 degrees
Magnitude: 4.15 Direction: 57.55 degrees
Magnitude: 13.94 Direction: 43.95 degrees
Magnitude: 16.66 Direction: -4.65 degrees
Magnitude: 6.24 Direction: -143.5 degrees
Magnitude: 4.49 Direction: -133.55 degrees
Magnitude: 12.07 Direction: 153.83 degrees
Magnitude: 11.31 Direction: -5.21 degrees
Magnitude: 18.41 Direction: 27.06 degrees
Magnitude: 5.26 Direction: 32.13 degrees
Magnitude: 3.74 Direction: 28.05 degrees
Magnitude: 9.9 Direction: -68.61 degrees
Magnitude: 6.86 Direction: 35.01 degrees
Magnitude: 11.85 Direction: 20.8 degrees
Magnitude: 12.82 Direction: -134.19 degrees
Magnitude: 41.1 Direction: -133.53 degrees
Magnitude: 15.84 Direction: -110.81 degrees
Magnitude: 13.54 Direction: -121.91 degrees
Magnitude: 6.93 Direction: -110.18 degrees
Magnitude: 1.11 Direction: -62.72 degrees
Magnitude: 1.92 Direction: -83.44 degrees
Magnitude: 15.11 Direction: -50.44 degrees
Magnitude: 3.05 Direction: -87.27 degrees
Magnitude: 8.23 Direction: -18.86 degrees
Magnitude: 4.12 Direction: 48.59 degrees
Magnitude: 15.01 Direction: 38.67 degrees
Magnitude: 3.38 Direction: -20.28 degrees
Magnitude: 6.22 Direction: -60.11 degrees
Magnitude: 2.53 Direction: -135.03 degrees
Name: Aryan Agarwal
CU240251800

```

Step 6 Repeat the experiment using the Farneback Dense Optical Flow algorithm and compare the tracking performance.

```

flow = cv2.calcOpticalFlowFarneback(
    old_gray, new_gray, None,
    0.5, 3, 15, 3, 5, 1.2, 0
)

magnitude, angle = cv2.cartToPolar(
    flow[...,0], flow[...,1]
)

hsv = np.zeros((*old_gray.shape, 3), dtype=np.uint8)
hsv[...,0] = angle * 180 / np.pi / 2
hsv[...,1] = 255
hsv[...,2] = cv2.normalize(magnitude, None, 0, 255, cv2.NORM_MINMAX)

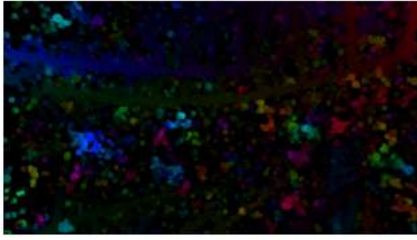
result = cv2.cvtColor(hsv, cv2.COLOR_HSV2RGB)

plt.figure(figsize=(4, 3))
plt.imshow(result)
plt.title("Farneback Optical Flow")
plt.axis("off")
plt.show()
plt.close()

print("Name: Aryan Agarwal")
print("CU240251800")

```

Farneback Optical Flow



Name: Aryan Agarwal
CU240251800

Step 7 Evaluate the impact of object speed, illumination changes, occlusion, and camera movement on tracking accuracy

```
print("Object speed: High speed may reduce tracking accuracy.")
print("Illumination: Sudden lighting changes can affect features.")
print("Occlusion: Hidden objects may cause tracking loss.")
print("Camera movement: Background motion can affect tracking accuracy.")

print("Name: Aryan Agarwal")
print("CU240251800")
```

Object speed: High speed may reduce tracking accuracy.
Illumination: Sudden lighting changes can affect features.
Occlusion: Hidden objects may cause tracking loss.
Camera movement: Background motion can affect tracking accuracy.
Name: Aryan Agarwal
CU240251800

Step 8 Analyze the tracked motion to estimate parameters such as object displacement, trajectory, and average speed.

```
distances = []

for a, b, s in zip(p0, p1, status):
    if s == 1:
        d = np.linalg.norm(b.ravel() - a.ravel())
        distances.append(d)

print("Average displacement:", round(np.mean(distances), 2))
print("Maximum displacement:", round(np.max(distances), 2))

print("Name: Aryan Agarwal")
print("CU240251800")
```

Average displacement: 9.88
Maximum displacement: 41.1
Name: Aryan Agarwal
CU240251800

Step 9 Compare the effectiveness of optical flow-based tracking with conventional frame-by-frame object detection techniques

```
print("Optical Flow:")
print("- Tracks motion between consecutive frames.")
print("- Fast for continuous motion tracking.")

print("\nObject Detection:")
print("- Detects objects independently in each frame.")
print("- Usually more computationally expensive.")

print("Name: Aryan Agarwal")
print("CU240251800")
```

Optical Flow:

- Tracks motion between consecutive frames.
- Fast for continuous motion tracking.

Object Detection:

- Detects objects independently in each frame.
- Usually more computationally expensive.

Name: Aryan Agarwal

CU240251800

- Step 10 Document the observations and discuss practical applications of optical flow in intelligent vision systems.

```
print("Observation: Optical flow successfully estimates object motion")
print("between consecutive video frames.")
```

```
print("Applications:")
print("1. Object tracking")
print("2. Video surveillance")
print("3. Autonomous vehicles")
print("4. Robot navigation")
print("5. Intelligent vision systems")
```

```
print("Name: Aryan Agarwal")
print("CU240251800")
```

Observation: Optical flow successfully estimates object motion
between consecutive video frames.

Applications:

1. Object tracking
2. Video surveillance
3. Autonomous vehicles
4. Robot navigation
5. Intelligent vision systems

Name: Aryan Agarwal

CU240251800